

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Grafikus megjelenítés és egyéb hasznos elemek

- Python-ban nagyon sok mindenre van előre megírt / elkészített modul.
 - `argparse` Parancssori argumentumok kezelésére. [7]
 - A modult az `import argparse` utasítás után tudjuk használni.
 - `csv` CSV (Comma-Separated Value) fájlok kezelésére. [12]
 - A modult az `import csv` utasítás után tudjuk használni.
 - `json` JSON (JavaScript Object Notation) fájlok kezelésére. [13]
 - A modult az `import json` utasítás után tudjuk használni.
 - `matplotlib` Függvények és adatsorok grafikus megjelenítésére. [14]
 - A modult az `import matplotlib` utasítás után tudjuk használni.
 - `numpy` Komplexebb matematikai számításokra (vektorok, mátrixok, ...). [15]
 - A modult az `import numpy` utasítás után tudjuk használni.
 - `networkx` Gráfokkal történő munkavégzésre. [16]
 - A modult az `import networkx` utasítás után tudjuk használni.

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólagos számítás
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A csv modul

CSV fájlok

- A Comma-Separated Values fájl egy olyan szöveges fájl, melynek tartalma egy táblázat.
 - A táblázat egy sora egy sor a fájlban.
 - Egy sor oszlopainak értékei egy szeparátor karakterrel (általában tabulátor, `,` vagy `;`) vannak elválasztva.
 - Egy cella értéke sztringhatároló jelek (általában `'` vagy `"`) közé lehet téve.
 - A táblázat (így a fájl) első sora oszlopfejléceket tartalmazhat.

f1.csv [1-11]:

```
"Name";"Started";"Finished";"Fastest Lap"
"ALB";13;14;"1:23.930"
"ALO";7;11;"1:23.063"
"BOT";12;16;"1:22.792"
"GAS";20;NC;"1:23.340"
"HAM";5;3;"1:22.153"
"HUL";11;13;"1:23.461"
"LEC";6;4;"1:22.182"
"MAG";15;15;"1:23.553"
"NOR";1;2;"1:21.712"
"OCO";19;18;"1:21.610"
```

points.csv [1-11]:

```
Name;Points
PIA;25
NOR;18
HAM;15
LEC;12
VER;10
SAI;8
PER;6
RUS;4
TSU;2
STR;1
```

A csv modul

Beolvasás listába, kiírás listából

- `reader(csvfile, dialect='excel', **fmtparams)`
A fájl sorait sztringek listájaként adja vissza.
- `writer(csvfile, dialect='excel', **fmtparams)`
Stringek listáját tudja megfelelő módon a csv egy soraként kiírni.



A csv modul

Példa – csv.reader, csv.writer

```
import csv
points = {1:25, 2:18, 3:15, 4:12, 5:10, 6:8, 7:6, 8:4, 9:2, 10:1}
drvpts = [None] * 10
with open('f1.csv', 'rt', newline='') as file:
    reader = csv.reader(file, delimiter=',')
    head = next(reader)          # read next row
    pos = head.index('Finished')
    drv = head.index('Name')
    for row in reader:          # iterate rows
        if row[pos].isnumeric():
            finished = int(row[pos])
            if finished <= 10:
                drvpts[finished-1] = (row[drv], points[finished])
with open('points.csv', 'wt', newline='') as file:
    writer = csv.writer(file, delimiter=';', lineterminator='\n')
    writer.writerow(['Name', 'Points'])
    for row in drvpts:
        writer.writerow(row)
```

f1.csv [1–5]:

```
"Name";"Started";"Finished";"Fastest Lap"
"ALB";13;14;"1:23.930"
"ALO";7;11;"1:23.063"
"BOT";12;16;"1:22.792"
"GAS";20;NC;"1:23.340"
```

points.csv [1–5]:

```
Name;Points
PIA;25
NOR;18
HAM;15
LEC;12
```

A csv modul

Beolvasás dictionary-be, kiírás dictionary-ból

- `DictReader(f, fieldnames=None, restkey=None, restval=None, dialect='excel', *args, **kwargs)`
A csv sorait dict-ként olvassa be.
- `DictWriter(f, fieldnames, restval='', extrasaction='raise', dialect='excel', *args, **kwargs)`
dict típusú értékeket képes soronként a csv fájlba írni.



A csv modul

Példa – csv.DictReader, csv.DictWriter

```
import csv
points = {1:25, 2:18, 3:15, 4:12, 5:10, 6:8, 7:6, 8:4, 9:2, 10:1}
with open('f1.csv', 'rt', newline='') as infile, open('points-dict.csv', 'wt', newline='') as outfile:
    reader = csv.DictReader(infile, delimiter=';')
    writer = csv.DictWriter(outfile, ['Name', 'Points'], delimiter=';', lineterminator='\n')
    writer.writeheader()
    for row in reader:
        # iterate rows
        if row['Finished'].isnumeric():
            finished = int(row['Finished'])
            if finished <= 10:
                newrow = {'Name': row['Name'], 'Points': points[finished]}
                writer.writerow(newrow)
```

f1.csv [1–10]:

```
"Name";"Started";"Finished";"Fastest Lap"
"ALB";13;14;"1:23.930"
"ALO";7;11;"1:23.063"
"BOT";12;16;"1:22.792"
"GAS";20;NC;"1:23.340"
"HAM";5;3;"1:22.153"
"HUL";11;13;"1:23.461"
"LEC";6;4;"1:22.182"
"MAG";15;15;"1:23.553"
"NOR";1;2;"1:21.712"
```

points-dict.csv [1–10]:

```
Name;Points
HAM;15
LEC;12
NOR;18
PER;6
PIA;25
RUS;4
SAI;8
STR;1
TSU;2
```

A json modul

JSON fájlok

- A JavaScript Object Notation szöveges fájl tartalma egy objektum leírása.
 - A fájlt leginkább egymásba ágyazott listák és dictionary-k struktúrájaként lehet elképzelni.
 - A listák és dictionary-k megadása nagyon hasonlít a Python szintaxisához ([,], {, }, : és , jelekkel), és nem érzékeny a whitespace karakterekre.
 - Egy érték lehet sztring, szám, true, false, null, lista vagy dictionary.
 - A dictionary-ben kulcs csak sztring lehet, amit " jel határol.

f1.json [1–9]:

```
[  
  { "Name": "ALB",  
    "Started": 13,  
    "Finished": 14,  
    "Fastest Lap": "1:23.930" },  
  { "Name": "ALO",  
    "Started": 7,  
    "Finished": 11,  
    "Fastest Lap": "1:23.063" },  
]
```

f1.json [74–82]:

```
{ "Name": "VER",  
  "Started": 3,  
  "Finished": 5,  
  "Fastest Lap": "1:20.908" },  
{ "Name": "ZHO",  
  "Started": 18,  
  "Finished": 19,  
  "Fastest Lap": "1:23.487" }  
]
```

A json modul

Beolvasás dictionary-be, kiírás dictionary-ból

- `json.load(fp, *, **kw)`
A json tartalmát dict-ként olvassa be.
- `json.dump(obj, fp, *, skipkeys=False, ensure_ascii=True, allow_nan=True, indent=None, sort_keys=False, **kw)`
dict típusú értékeket képes json fájlba írni.



A json modul

Példa – json.load, json.dump

```
import json
points = {1:25, 2:18, 3:15, 4:12, 5:10, 6:8, 7:6, 8:4, 9:2, 10:1}
with open('f1.json', 'rt') as file:
    data = json.load(file)
    for row in data:
        finished = row['Finished']
        if type(finished) is int:
            if finished <= 10:
                row['Points'] = points[finished]
with open('points.json', 'wt') as file:
    json.dump(data, file, indent=4)
```

f1.json [18–29]:

```
{ "Name": "HAM",
  "Started": 5,
  "Finished": 3,
  "Fastest Lap": "1:22.153" },
{ "Name": "HUL",
  "Started": 11,
  "Finished": 13,
  "Fastest Lap": "1:23.461" },
{ "Name": "LEC",
  "Started": 6,
  "Finished": 4,
  "Fastest Lap": "1:22.182" },
```

points.json [26–38]:

```
{
  "Name": "HAM",
  "Started": 5,
  "Finished": 3,
  "Fastest Lap": "1:22.153",
  "Points": 15
},
{
  "Name": "HUL",
  "Started": 11,
  "Finished": 13,
  "Fastest Lap": "1:23.461"
},
```

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció**
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Parancssori argumentumok kezelése

Az argparse modul

- Parancssori argumentumok definiálására és automatikus kezelésére szolgál.
- Először létre kell hozni egy parancssor-elemzőt:

```
parser = argparse.ArgumentParser(description="Help_ szöveg")
```

- `ArgumentParser(description=None, epilog=None, prefix_chars='-', ...)`

`description` A help elején megjelenő szöveg.

`epilog` A help végén megjelenő szöveg.

`prefix_chars` A kapcsolók prefix karaktere.

Parancssori argumentumok kezelése

Az argparse modul

- Utána hozzá kell adni argumentumokat és kapcsolókat:

```
parser.add_argument('command', help='Az első argumentum')
parser.add_argument('arguments', nargs='*', default=[],
                    help='A többi argumentum egy listába gyűjtve')
parser.add_argument('-v', '--verbose', dest='printing', default=False,
                    action='store_true', help='Egy kapcsoló')
```

- `add_argument(*args, action='store', nargs=None, default=None, type=str, choices=None, required=False, help=None, dest=None, ...)`

Parancssori argumentumok és kapcsolók hozzáadása a programhoz, és ezek feldolgozási módjának megadása.

Parancssori argumentumok kezelése

Az argparse modul

- `add_argument(*args, action='store', nargs=None, default=None, type=str, choices=None, required=False, help=None, dest=None, ...)`

`*args` A sorrendi paraméter vagy a kapcsoló neve(i).

`action` Mi csináljunk? `'store'`, `'store_true'`, `'store_false'`, `'append'`, `'extend'`, ...

`nargs` A feldolgozandó argumentumok száma. `N`, `'?'`, `'*'`, `'+'`

`default` Alapértelmezett érték, ha az argumentum hiányzik.

`type` Az argumentum típusa, amire konvertálni kell.

`choices` Az argumentum elfogadott értékei, listában.

`required` Kötelező-e?

`help` Az argumentum leírása.

`dest` Milyen néven lesz elérhető az argumentum értéke a feldolgozás után?

Parancssori argumentumok kezelése

Az argparse modul

- Ezután jöhet az argumentumok feldolgozása:

```
args = parser.parse_args()
```

- `parse_args()`: Feldolgozza a parancssori argumentumokat.
- Végül pedig a használata:

```
if args.printing:  
    print(args.command)  
    for a in args.arguments:  
        print(a)
```

Grafikus megjelenítés

A matplotlib modul

- Függvények és egyéb grafikonok megjelenítésére szolgál.
- Először létre kell hozni egy ábrát:

```
matplotlib.pyplot.figure(figsize=(16,9))
```

- `figure(figsize=None, dpi=None, ...)`

`figsize` Az ábra mérete, inch-ben (alapértelmezett $6.4'' \times 4.8''$).

`dpi` Felbontás, *dots per inch* (alapértelmezett: 100 dpi).

Grafikus megjelenítés

A matplotlib modul

- Utána meg kell adni a kirajzolandó függvény pontjait:

```
matplotlib.pyplot.plot([0,1,2,3,4,5,6],[-9,-4,-1,0,1,4,9], 'D:r')
```

- `plot(x, y, format=None, ...)`

`x` Adatpontok x koordinátái.

`y` Adatpontok y koordinátái.

`format` Az adatsor kirajzolásának paraméterei, sztringként:

- pontstílus: `'.'`, `'o'`, `'s'`, `'x'`, `'D'`, ...
- vonalstílus: `'-'`, `'--'`, `'-.'`, `':'`
- szín: `'w'`, `'r'`, `'y'`, `'g'`, `'c'`, `'b'`, `'m'`, `'k'`
- Vagy csak szín: `'green'`, `'#efbf04'`, ...

Grafikus megjelenítés

A matplotlib modul

- Végül meg kell jeleníteni az ábrát:

```
matplotlib.pyplot.show()
```

- `show()`: Megjeleníti az ábrát (a q billentyűvel lehet kilépni).
- Vagy megjelenítés helyet el is menthetjük képként:

```
matplotlib.pyplot.savefig('abra.png', transparent=True)
```

- `savefig(fname, transparent=False, format=None, ...)`

Adott fájl névvel képként elmenti az ábrát.

`fname` A képfájl neve.

`transparent` Átlátszó kép mentése.

`format` A képformátum ('png', 'pdf', 'svg', ...).

Egyenirányító feszültségének megjelenítése

Problémafelvetés és specifikáció

- Problémafelvetés
 - Rajzoljuk fel egy egyenirányító kimenetének idő-feszültség diagramját.
- Specifikáció
 - A program parancssorból kaphasson bizonyos paramétereket: bemeneti feszültség (V), bemeneti frekvencia (Hz), kimeneti oldali fogyasztó ellenállás ($k\Omega$), a kondenzátor Ohmikus ellenállása ($k\Omega$), a kondenzátor kapacitása (μF), az ábrázolandó időintervallum (ms).
 - Lehesse kiválasztani, hogy egy vagy kétutas egyenirányítót használunk.
 - A kimeneti feszültséget elég numerikusan közelíteni (kellő pontossággal).
 - A bemeneti, egyenirányított, kondenzátor és kimeneti feszültséget is idő-feszültség diagramon ábrázoljuk.

Egyenirányító feszültségdiagrammja [1/4]

egyeniranyito.py [1-24]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Egyenirányító kimeneti feszültségének ábrázolása
9"""
10import argparse
11from math import sin
12from math import pi
13from math import e
14import numpy as np
15import matplotlib.pyplot as plt
16
17def input_feszultseg(t, *, U, f):
18    return U * sin(t*f*2.0*pi)
19
20def egyeniranyito_1(U):
21    return U if U >= 0.0 else 0.0
22
23def egyeniranyito_2(U):
24    return abs(U)
```

Egyenirányító feszültségdiagrammja [2/4]

egyeniranyito.py [26–39]

```
26 def kondenzator(U_t, *, t, C, Rc, R):
27     if C == 0.0:
28         return U_t
29     C_t = [0.0] * len(U_t)
30     dt = t / (len(U_t) - 1)
31     cc = e ** (- dt / (Rc * C)) if Rc > 0.0 else 0.0
32     cd = e ** (- dt / ((R + Rc) * C)) if (R + Rc) > 0.0 else 0.0
33     for i in range(1, len(U_t)):
34         U_diff = U_t[i] - C_t[i-1]
35         if U_diff >= 0:
36             C_t[i] = C_t[i-1] + (U_diff) * (1 - cc)
37         else:
38             C_t[i] = C_t[i-1] + (U_diff) * (1 - cd)
39     return C_t
```

Egyenirányító feszültségdiagrammja [3/4]

egyeniranyito.py [41–56]

```
41 def main():
42     parser = argparse.ArgumentParser(description = 'Egyenirányító')
43     parser.add_argument('-U', default= 10.0, type=float, help='Input AC feszültség(V)')
44     parser.add_argument('-f', default= 50.0, type=float, help='Input AC frekvencia(Hz)')
45     parser.add_argument('-R', default= 0.01, type=float, help='Áramkör DC oldali fogyasztó ellenállás( $k\Omega$ )')
46     parser.add_argument('-Rc', default= 0.001, type=float, help='A kondenzátor Ohmikus ellenállása( $k\Omega$ )')
47     parser.add_argument('-C', default=4700.0, type=float, help='A kondenzátor kapacitása( $\mu F$ )')
48     parser.add_argument('-t', default= 200.0, type=float, help='Ábrázolandó időintervallum(ms)')
49     parser.set_defaults(egyutas=True)
50     parser.add_argument('-1', dest='egyutas', action='store_true', help='Egyutas egyenirányítás')
51     parser.add_argument('-2', dest='egyutas', action='store_false', help='Graetz-hidas egyenirányítás')
52     args = parser.parse_args()
53     args.R *= 1.0e3
54     args.Rc *= 1.0e3
55     args.C *= 1.0e-6
56     args.t *= 1.0e-3
```

Egyenirányító feszültségdiagrammja [4/4]

egyeniranyito.py [58–79]

```
58 x_axis = np.linspace(0.0, args.t, int(40 * args.t * args.f) + 1)
59
60 feszultseg = np.vectorize(input_feszultseg)
61 y_inp = feszultseg(x_axis, U=args.U, f=args.f)
62
63 egyeniranyito = np.vectorize(egyeniranyito_1 if args.egyutas else egyeniranyito_2)
64 y_mid = egyeniranyito(y_inp)
65
66 y_cap = kondenzator(y_mid, t=args.t, C=args.C, Rc=args.Rc, R=args.R)
67
68 y_out = [max(u, c) for u, c in zip(y_mid, y_cap)]
69
70 plt.figure(figsize=(16,9))
71 plt.plot(x_axis, y_inp, 'gray', linewidth=1)
72 plt.plot(x_axis, y_mid, 'blue', linewidth=2)
73 plt.plot(x_axis, y_cap, 'green', linewidth=3)
74 plt.plot(x_axis, y_out, 'red', linewidth=4)
75
76 plt.show()
77
78 if __name__ == "__main__":
79     main()
```

Röppálya ábrázolása

A probléma

- Eldobunk egy testet, milyen röppályán mozog?
- Csak egyszerűen: sík terepen, légellenállás nélkül, pontszerű testtel, közvetlenül a felszínről dobjuk.
- Egyenletes időközönként figyeljük, hogy hol van a test.
- Emlékszünk még?
 - $v_{0,x} = v_0 \cdot \cos \alpha$
 - $v_{0,y} = v_0 \cdot \sin \alpha$
 - $\Delta s_x = v_x \cdot \Delta t$
 - $\Delta s_y = v_y \cdot \Delta t (+ \frac{g}{2} \cdot \Delta t^2)$
 - $\Delta v_x = 0$
 - $\Delta v_y = g \cdot \Delta t$
- Számoljunk fixen m/s -ban.
- Ábrázoljuk a röppályát!

- A röppálya szimulációja már készen van, azt fejlesztjük tovább.



Röppálya szimulációja [1/2]

roppalya_szimulacio.py (v1.0) [1–13]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-07-16
6cc-by-nc-nd
7
8Ferdehajítás röppálya szimulációja
9"""
10from math import sin
11from math import cos
12from roppalya import G
13from roppalya import deg_to_rad
```

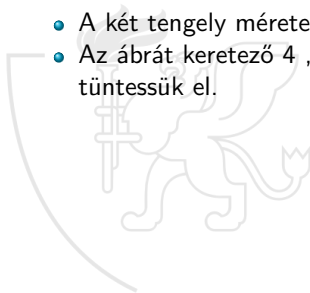
Röppálya szimulációja [2/2]

roppalya_szimulacio.py (v1.0) [15–30]

```
15 def main():
16     v0 = float(input("Kezdősebesség(m/s)? "))
17     alfa = float(input("Szög(fok)? "))
18     dt = float(input("Delta_t(s)? "))
19     t = 0.0
20     vx, vy = v0 * cos(deg_to_rad(alfa)), v0 * sin(deg_to_rad(alfa))
21     sx, sy = 0.0, 0.0
22     while sy >= 0.0:
23         print(f"{t:.3f}s: p({sx:.3f}, {sy:.3f}); v={vx:.3f}, {vy:.3f}")
24         sx += vx * dt
25         sy += vy * dt - G / 2.0 * dt ** 2
26         vy -= G * dt
27         t += dt
28
29 if __name__ == "__main__":
30     main()
```

- Amit ábrázolunk, az tulajdonképpen egy függvény lesz, amely adott x irányú elmozduláshoz hozzárendeli az y irányú elmozdulást.
- A test pozícióit adott Δt időközönként már kiszámoltuk, kiírtattuk.
- Most ezeket össze kellene gyűjteni egy listában.
- Annyi különbség még lesz, hogy eddig azokat a pozíciókat írtattuk ki, amikor „levegőben” volt a test (beleértve, hogy éppen érinti a talajt), most viszont úgy kellene összegyűjteni a pozíciókat, hogy az utolsó pont az a talajérintés vagy a talajba fúródás legyen. (Így szebben néz majd ki az ábra, nem a levegőben áll meg a test.)

- A `matplotlib` modul segítségével ábrázoljuk a „függvényünket”, ami x irányú elmozduláshoz hozzárendeli az y irányú elmozdulást.
- A függvényünket a korábban kiszámolt véges számú értékpár megadásával közelítjük.
- Beállítások segítségével tegyük szebbé az ábrát:
 - Válasszunk 16:9-es képarányt.
 - A két tengely méretezése legyen azonos.
 - Az ábrát keretező 4 „tengely” közül kettő a $(0,0)$ adatpontra illeszkedjen, a másik kettőt tüntessük el.



Röppálya ábrázolása [1/2]

roppalya_szimulacio.py (v2.0) [1-14]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTIK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-07-16
6cc-by-nc-nd
7
8Ferdehajítás, röppálya, ábrázolása
9"""
10from math import sin
11from math import cos
12import matplotlib.pyplot as plt
13from roppalya import G
14from roppalya import deg_to_rad
```

Röppálya ábrázolása [2/2]

roppalya_szimulacio.py (v2.0) [16–42]

```
16 def main():
17     v0 = float(input("Kezdősebesség (m/s)? "))
18     alfa = float(input("Szög (fok)? "))
19     dt = float(input("Delta t (s)? "))
20     vx, vy = v0 * cos(deg_to_rad(alfa)), v0 * sin(deg_to_rad(alfa))
21     sx, sy = 0.0, 0.0
22     s = [(sx, sy)]
23     while sy >= 0.0:
24         sx += vx * dt
25         sy += vy * dt - G / 2.0 * dt ** 2
26         s.append((sx, sy))
27         vy -= G * dt
28
29     x_axis = [x for (x, _) in s]
30     y_axis = [y for (_, y) in s]
31     plt.figure(figsize=(16,9))
32     plt.plot(x_axis, y_axis)
33     ax = plt.gca()
34     ax.set_aspect('equal')
35     ax.spines['left'].set_position(('data', 0.0))
36     ax.spines['bottom'].set_position(('data', 0.0))
37     ax.spines['top'].set_visible(False)
38     ax.spines['right'].set_visible(False)
39     plt.show()
40
41 if __name__ == "__main__":
42     main()
```

Függvény határozott integráljának megjelenítése

Problémafelvetés és specifikáció

- Problémafelvetés

- Adott függvény határozott integrálját közelítsük egy adott intervallumon a felhasználó által megadott számú részre osztva az intervallumot, és jelenítsük is meg!

- Specifikáció

- A probléma inputja az integrálandó függvény neve, a , b valós számok, az intervallum végpontjai, n , a közelítéshez használandó részek száma.
- Az inputot a parancssorból vegye a program.
- Az output három valós szám: a megadott függvény értéke a és b pontokban, valamint a határozott integrál értéke.
- A program jelenítse is meg a függvényt és az integrálját.

Függvény határozott integráljának megjelenítése [1/3]

fgvint_show.py [1-24]

```
1#!/usr/bin/env python3
2"""
3Programozás_Alapjai_előadás_példaprogram
4SZTE_TTIK, Informatikai_Intézet, Szoftverfejlesztés_Tanszék
5Gergely_Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Függvény_határozott_integráljának_kiszámítása
9"""
10import argparse
11from math import sin
12from math import cos
13from math import tan
14from math import exp
15from sys import stderr
16import numpy as np
17import matplotlib.pyplot as plt
18from matplotlib.patches import Polygon
19
20def expx(x):
21    return exp(x) / x
22
23def sin2x(x):
24    return sin(2.0 * x)
```

Függvény határozott integráljának megjelenítése [2/3]

fgvint_show.py [26–52]

```
26 ISMERT_FUGGVENYEK = {
27     "sin": sin,
28     "tan": tan,
29     "cos": cos,
30     "exp": exp,
31     "sin2x": sin2x
32 }
33
34 def trapez(f, a, b, n):
35     h = (b - a) / n
36     area = 0.0
37     for i in range(1, n):
38         area += f(a + i * h)
39     return (area + (f(a) + f(b)) / 2.0) * h
40
41 def main():
42     global ISMERT_FUGGVENYEK
43     parser = argparse.ArgumentParser(description = 'Függvény_határozott_integrálját_számitó_program')
44     parser.add_argument('fgv', metavar='FÜGGVÉNY', default=None, type=str,
45                         help='Az_integrálandó_függvény', choices = ISMERT_FUGGVENYEK.keys())
46     parser.add_argument('a', metavar='A', default=None, type=float,
47                         help='Az_integrál_intervallumának_alsó_végpontja')
48     parser.add_argument('b', metavar='B', default=None, type=float,
49                         help='Az_integrál_intervallumának_felső_végpontja')
50     parser.add_argument('res', metavar='N', default=None, type=int,
51                         help='Az_intervallum_felbontása')
52     args = parser.parse_args()
```

Függvény határozott integráljának megjelenítése [3/3]

fgvint_show.py [54–77]

```
54 f = ISMERT_FUGGVENYEK[args.fgv]
55 try:
56     print("Az {f}(x) függvény határozott integrálja a(z) [{a:.3},{b:.3}] intervallumon {fiab:.3}.".
57           format(f=args.fgv, a=args.a, b=args.b, fiab=trapez(f, args.a, args.b, args.res)))
58
59     f = np.vectorize(f)
60
61     x_int = np.linspace(args.a, args.b, args.res+1)
62     y_int = f(x_int)
63     pnts = [(args.a, 0), *zip(x_int, y_int), (args.b, 0)]
64     pgon = Polygon(pnts, facecolor='red', alpha=0.2, edgecolor='red')
65     plt.gca().add_patch(pgon)
66
67     x_fgv = np.linspace(args.a, args.b, 20 * args.res)
68     y_fgv = f(x_fgv)
69     plt.plot(x_fgv, y_fgv, 'blue', linewidth=2)
70
71     plt.show()
72 except ZeroDivisionError:
73     print("A függvény az [{a:.3},{b:.3}] intervallum valamely pontjában nem értelmezett!".
74           format(a=args.a, b=args.b), file=stderr)
75
76 if __name__ == "__main__":
77     main()
```

Klikkek megjelenítése

Problémafelvetés és specifikáció

- Problémafelvetés

- Egy közösségben az emberek közötti barátsági kapcsolat alapján meg kell határoznunk és meg kell jelenítenünk a klikkeket!

- Specifikáció

- Az emberek számát (N) előre bekérjük, és az embereket az $1, \dots, N$ számokkal jelöljük.
- A program inputja i, j számpárok sorozata ($1 \leq i, j \leq N$), ami az i . és j . sorszámú ember barátságát jelenti. A sorozat (és az input) végét egy üres sor jelöli.
- A program gráfként jelenítse meg a közösséget.



Klikkek kirajzolása [1/2]

klikk_show.py [1-27]

```
1#!/usr/bin/env python3
2"""
3Programozás_Alapjai_előadás_példaprogram
4SZTE_TTIK, Informatikai_Intézet, Szoftverfejlesztés_Tanszék
5Gergely_Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Klikkek_meghatározása_és_megjelenítése.
9"""
10import matplotlib.pyplot as plt
11import networkx as nx
12
13def main():
14    n = int(input("Hány_ember_van? "))
15    klikkek = [set([i]) for i in range(1, n+1)]
16
17    while (r := input("Kik_ismerik_egymást_(a,b)? ")):
18        (a, b) = [int(i) for i in r.split(',')]
19        for ai in range(0, len(klikkek)):
20            if a in klikkek[ai]:
21                break
22        for bi in range(0, len(klikkek)):
23            if b in klikkek[bi]:
24                break
25        if ai != bi:
26            klikkek[ai].update(klikkek[bi])
27        del klikkek[bi]
```

Klikkek kirajzolása [2/2]

klikk_show.py [29–43]

```
29 edges = []
30 for k in klikkek:
31     edges.extend([(i, j) for i in k for j in k if i < j])
32 gr = nx.Graph()
33 gr.add_nodes_from(range(1, n+1))
34 gr.add_edges_from(edges)
35 pos = nx.spring_layout(gr)
36 nx.draw_networkx_nodes(gr, pos, node_color = 'b', node_size = 500, alpha = 1)
37 nx.draw_networkx_labels(gr, pos, font_color='w')
38 nx.draw_networkx_edges(gr, pos, edge_color='b')
39 plt.axis('off')
40 plt.show()
41
42 if __name__ == "__main__":
43     main()
```